02-gradient-descent_filled

April 12, 2026

1 CSE 25 - Introduction to Artificial Intelligence

1.1 Week 3 Tuesday: Gradient descent

Context (from last class):
Last time, we fit a line by manually adjusting parameters and measuring error. We saw that different parameter values lead to different errors.

Today, we focus on the question: **How can a system automatically change its parameters to reduce error?**

Learning Objectives: By the end of today's class, you will be able to:

- Describe several ways of measuring error of a model and compare their properties.
- Interpret error as a function of a model parameter.
- Explain how changing a parameter affects error.
- Describe brute-force search as a way to find better parameters.
- Explain, at a high level, how slope (rate of change) tells us which direction to adjust a parameter.
- Describe the difference between local and global optima.
- Connect these ideas to the motivation behind gradient descent.
- Define simple and multiple linear regression as fitting a line or hyperplane to predict target values from input features.

Instructions

Use your copy of this notebook on Datahub and complete it during class. Work through the cells below **in order**. You may discuss with your neighbors, but make sure you understand each step yourself.

SUBMISSION: When finished, download it as a PDF and upload it to Gradescope under `In-Class - Week 3 Tuesday` to receive credit. To download it as a PDF, on DataHub go to `File -> Save and Export Notebook As -> PDF`.

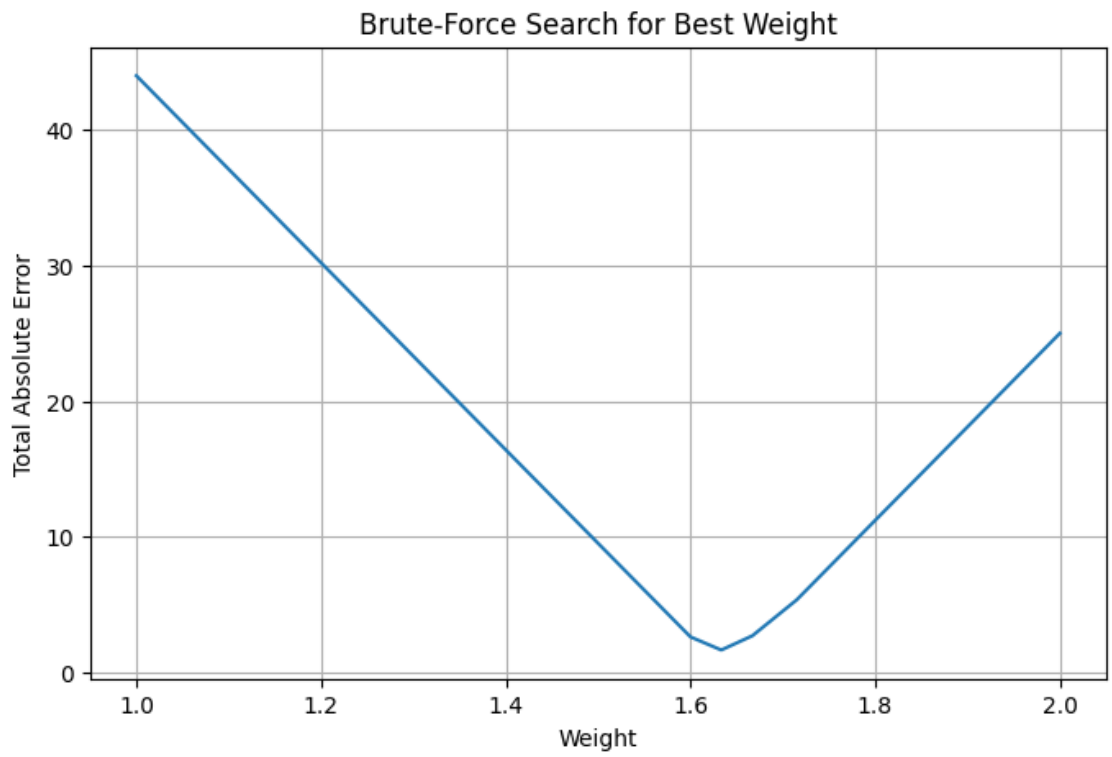
1.1.1 Review

Last time, we used brute-force search to find the best weight with bias fixed to 0 for a model fitting the input-output relationship of miles to kilometers and the best weight and bias for a model fitting the input-output relationship of degrees Fahrenheit to degree Celsius. In the brute-force search approach, we kept track of the values of the model parameters that give the smallest total absolute error.

[1]:
```python
# Import the matplotlib library for plotting
import matplotlib.pyplot as plt

# Data for miles and corresponding kilometers
# Stored in lists
miles = [0, 7, 12, 20, 30] # x-axis values (INPUT DATA)
km = [0, 12, 20, 32, 49] # y-axis values (OUTPUT DATA)

def get_predictions_v2(input_values, w, b):
    '''
    input_values: list of input values
    w: weight (slope)
    b: bias (intercept)

    Complete the function that calculates the predicted output values using the␣
 ↪weight and bias.
    Return a list of predicted values.
    '''
    # Initialize an empty list (predicted_values) to store predicted values
    predicted_values = []

    # Calculate predicted values for each input value by multiplying with␣
 ↪conversion factor and append it to predicted_values list
    for input_value in input_values:
        predicted_values.append(input_value * w + b)

    # Return the list of predicted values
    return predicted_values

def get_pointwise_absolute_error(actual_values, predicted_values):
    '''
    actual_values: list of actual output values
    predicted_values: list of predicted output values

    Complete the function that calculates the absolute error between predicted␣
 ↪and actual values for each value and returns a list of these absolute errors.
    '''
    # Calculate the error = actual - predicted for each point and store it in␣
 ↪the list
    # Using the List comprehension with zip function approach
    return [abs(actual - predicted) for actual, predicted in zip(actual_values,␣
 ↪predicted_values)]

# Brute-force search for the best weight (conversion factor)
# with bias fixed to 0

b = 0.0
```

```python
best_weight = None # Initialize best weight
lowest_error = float('inf') # Initialize to a very high value

weights = [] # To store weights tried
total_error_list = [] # To store total errors for each weight

# Try weights from 1.0 to 2.0 in steps of 0.1
w = 1.0
step_size = 0.001 # What happens if you change this to 1 or 0.01 or 0.001?

while w <= 2.0:
    predicted = get_predictions_v2(miles, w, b)
    errors = get_pointwise_absolute_error(km, predicted)
    total_error = sum(errors)
    total_error_list.append(total_error) # for the plot
    weights.append(w) # for the plot
    if total_error < lowest_error:
        lowest_error = total_error
        best_weight = w
    w += step_size

print(f'Best weight (conversion factor): {best_weight:.4f}')
print(f'Lowest total error: {lowest_error:.2f}')

plt.figure(figsize=(8,5))

# Changed the plot from scatter to line plot to visualize total error vs weight
plt.plot(weights,  total_error_list)

plt.xlabel("Weight")
plt.ylabel("Total Absolute Error")
plt.title("Brute-Force Search for Best Weight")
plt.grid()
plt.show()
```

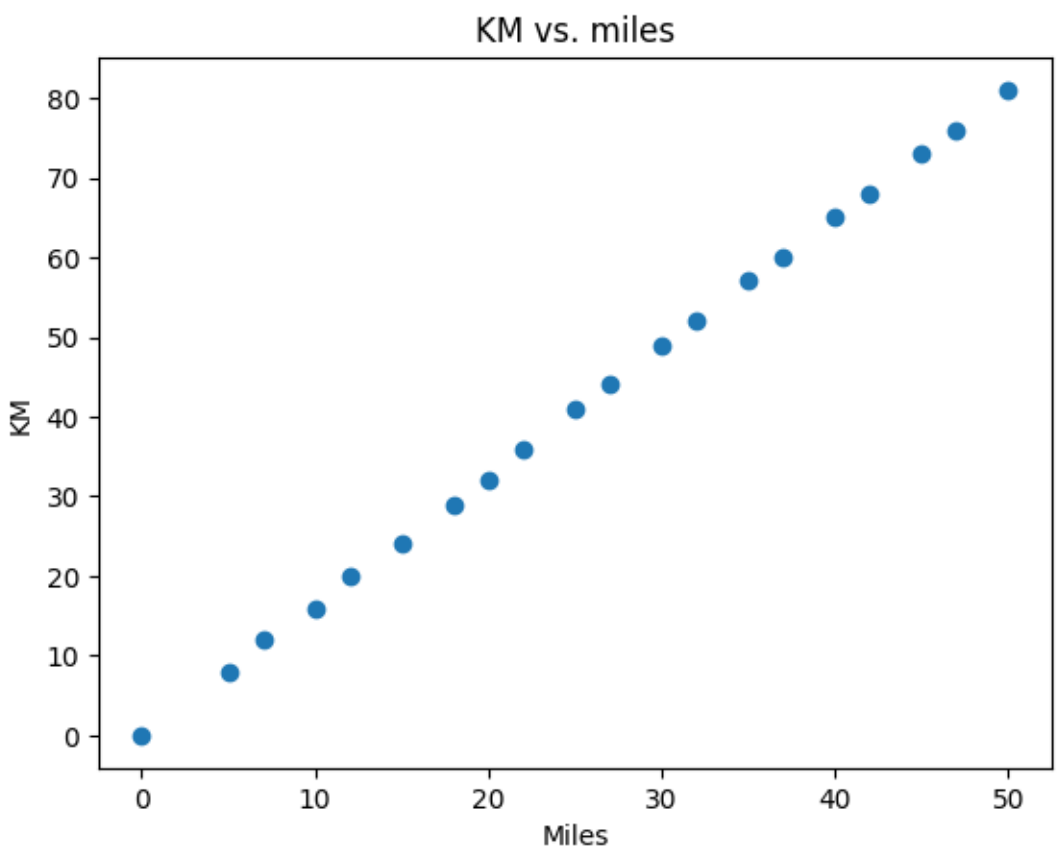
```
Best weight (conversion factor): 1.6330
Lowest total error: 1.64
```



Problem: Total error depends on dataset size! Let's say we are given many more data points - what would happen to the total error?

[2]:
```
miles_many = [0, 5, 7, 10, 12, 15, 18, 20, 22, 25, 27, 30, 32, 35, 37, 40, 42,␣
↪45, 47, 50]
km_many = [0, 8, 12, 16, 20, 24, 29, 32, 36, 41, 44, 49, 52, 57, 60, 65, 68,␣
↪73, 76, 81]

# Create a scatter plot for larger dataset
plt.scatter(miles_many, km_many)
plt.xlabel("Miles")
plt.ylabel("KM")
plt.title("KM vs. miles")
plt.show()

# Use the best_weight and b from previous cells to get the total error on the␣
↪original points
predicted_original = get_predictions_v2(miles, best_weight, b)
errors_original = get_pointwise_absolute_error(km, predicted_original)
total_error_original = sum(errors_original)
```

```
# Use best_weight and b from previous cells to get the total error on more data␣
↪points
# YOUR CODE HERE
# DELETE FROM THIS FOR RELEASED VERSION
predicted_many = get_predictions_v2(miles_many, best_weight, b)
errors_many = get_pointwise_absolute_error(km_many, predicted_many)
total_error_many = sum(errors_many)
# DELETE UNTIL THIS FOR RELEASED VERSION


print(f'Total error (original points): {total_error_original:.2f}')
print(f"Total error (many more points): {total_error_many:.2f}")
```




```
Total error (original points): 1.64
Total error (many more points): 6.99
```

Why does the total error increase with more data points? Notice that even when our model fits the data just as well, the total error becomes much larger when we have more data points. This is because the total error is simply the sum of all individual errors, so adding more points (even with similar errors per point) will increase the total.

This means that total error depends on the size of the dataset, not just how well the model fits. To more fairly compare models or datasets of different sizes, we need a metric that is independent of the number of data points.

A common solution is to use the **average** error per point, called the **`Mean Absolute Error (MAE)`**. This gives us a normalized measure of error that is easier to compare across datasets. The MAE is calculated as:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

where: - n is the number of data points - y_i is the actual value of the i th data point - $\hat{y}_i$ is the predicted value of the i th data point - the Sigma notation means summing over all data points

Quick check: what would you need to change about the expression for MAE to give the total absolute error from before?

[3]:
```
# Complete the function that calculates the Mean Absolute Error (MAE) between␣
↪predicted and actual values.

def get_mean_absolute_error(actual_values, predicted_values):
    '''
    actual_values: list of actual output values
    predicted_values: list of predicted output values

    Complete the function that calculates the Mean Absolute Error (MAE) between␣
↪predicted and actual values.
    Return the MAE.
    '''
    # From scratch:
    # Loop through each value in actual_values and predicted_values
    # to calculate the absolute error and store it in the list, then
    # divide by the number of values
    # Or: use the previously defined functions
    # YOUR CODE HERE
    return  (sum(get_pointwise_absolute_error(actual_values,␣
↪predicted_values)))*1.0 / len(actual_values)
```

[4]:
```
# Testing the function calculating MAE

# Test 1: Simple integers
act1 = [10, 20, 30]
pre1 = [12, 18, 25]
# Calculation: (|10-12| + |20-18| + |30-25|) / 3 = (2 + 2 + 5) / 3 = 3.0
assert get_mean_absolute_error(act1, pre1) == 3.0
print(f"Test 1 Results: {get_mean_absolute_error(act1, pre1)}")

# Test 2: Perfect match (Error should be 0)
```

```
act2 = [1.5, 2.5, 3.5]
pre2 = [1.5, 2.5, 3.5]
# Calculation: (|1.5-1.5| + |2.5-2.5| + |3.5-3.5|) / 3 = (0 + 0 + 0) / 3 = 0
assert get_mean_absolute_error(act2, pre2) == 0
print(f"Test 2 Results: {get_mean_absolute_error(act2, pre2)}")

# Test 3: Large differences
act3 = [100, 0]
pre3 = [0, 100]
# Calculation: (|100-0| + |0-100|) / 2 = (100 + 100) / 2 = 100.0
assert get_mean_absolute_error(act3, pre3) == 100
print(f"Test 3 Results: {get_mean_absolute_error(act3, pre3)}")
```

```
Test 1 Results: 3.0
Test 2 Results: 0.0
Test 3 Results: 100.0
```

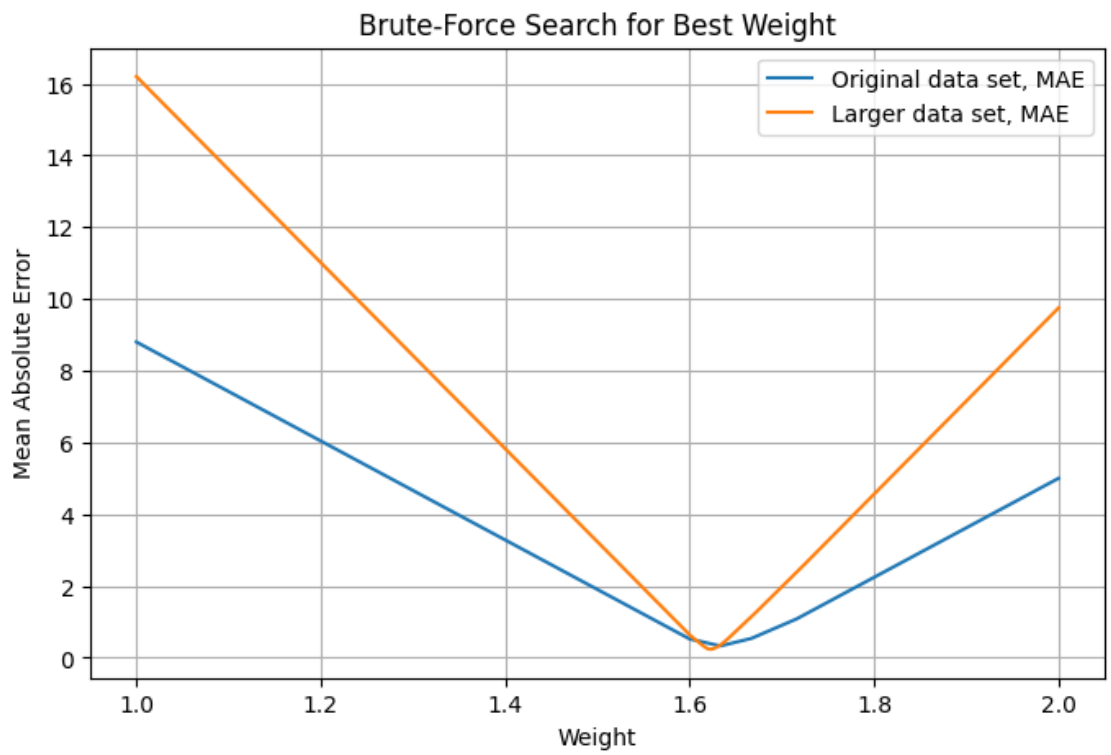
Let's run our brute force search again on both the original and the bigger data set, but this time, we will use the MAE intead.

[5]:
```
# Brute-force search for the best weight (conversion factor)
# with bias fixed to 0, using Mean Absolute Error as the error metric

# Original data set
best_weight_original = None # Initialize best weight
lowest_error_original = float('inf') # Initialize to a very high value
# Larger data set
best_weight_many = None # Initialize best weight
lowest_error_many = float('inf') # Initialize to a very high value

weights = [] # To store weights tried
mae_list_original = [] # To store mean absolute errors for each weight for␣
↪original data set
mae_list_many = [] # To store mean absolute errors for each weight for much␣
↪bigger data set

w = 1.0
b = 0.0
step_size = 0.001

while w <= 2.0:
    # Predict output values for each data point given current model
    predicted_original = get_predictions_v2(miles, w, b)
    predicted_many = get_predictions_v2(miles_many, w, b)
    # Calculate mean absolute error relative to data sets and current model
    mae_original = get_mean_absolute_error(km, predicted_original)
    mae_many = get_mean_absolute_error(km_many, predicted_many)
    # For the plot, update the weights list and the list of mean absolute errors
```

```
    mae_list_original.append(mae_original)
    mae_list_many.append(mae_many)
    weights.append(w)
    if mae_original < lowest_error_original:
        lowest_error_original = mae_original
        best_weight_original = w
    if mae_many < lowest_error_many:
        lowest_error_many = mae_many
        best_weight_many = w
    w += step_size

print(f'Best weight (conversion factor) with original data :␣
 ↪{best_weight_original:.4f}')
print(f'Lowest mean absolute error with original data: {lowest_error_original:.
 ↪2f}')

idx = weights.index(best_weight_original)
print(f'Mean absolute error for larger data set using model parameters\n from␣
 ↪brute-force search using original data : {mae_list_many[idx]:.2f}')

print(f'\nBest weight (conversion factor) with larger data set:␣
 ↪{best_weight_many:.4f}')
print(f'Lowest mean absolute error with larger data set: {lowest_error_many:.
 ↪2f}')
```

```
Best weight (conversion factor) with original data : 1.6330
Lowest mean absolute error with original data: 0.33
Mean absolute error for larger data set using model parameters
 from brute-force search using original data : 0.35

Best weight (conversion factor) with larger data set: 1.6220
Lowest mean absolute error with larger data set: 0.24
```

[6]:
```
plt.figure(figsize=(8,5))
plt.plot(weights,  mae_list_original, label="Original data set, MAE")
plt.plot(weights,  mae_list_many, label ="Larger data set, MAE")
plt.xlabel("Weight")
plt.ylabel("Mean Absolute Error")
plt.title("Brute-Force Search for Best Weight")
plt.grid()
plt.legend()
plt.show()
```



Compare the two graphs? `YOUR ANSWER HERE`

Describe how the graph with Total Absolute Error (instead of Mean Absolute Error) on the y axis would look compared to these graphs. `YOUR ANSWER HERE`

1.1.2 Mean Squared Error (MSE)

`Mean Squared Error (MSE)` is another common way to measure error. It is defined as:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

where: - n is the number of data points - y_i is the actual value of the i th data point - $\hat{y}_i$ is the predicted value of the i th data point - the Sigma notation means summing over all data points

Why might we prefer MSE over MAE? - MSE is a **smooth, differentiable function**, which makes it easier to use with a popular family of optimization methods. These and other mathematical properties of MSE make it easier to analyze and work with in many machine learning algorithms. - MSE **penalizes larger errors more heavily** (because errors are squared), which can be useful if we want to avoid large mistakes.

For comparison:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

[7]:
```
# Complete the function that calculates the Mean Squared Error (MSE) between␣
↪predicted and actual values.

def get_mean_squared_error(actual_values, predicted_values):
    '''
    actual_values: list of actual output values
    predicted_values: list of predicted output values

    Complete the function that calculates the Mean Squared Error (MSE) between␣
↪predicted and actual values.
    Return the MSE.
    '''
    # Loop through each value in actual_values and predicted_values
    # to calculate the absolute error and store it in the list, then
    # divide by the number of values
    # Or: use the previously defined functions
    # YOUR CODE HERE

    # Initialize list to store squared errors
    sq_errors = []
    # Loop through each value and accumulate the squared error
    for act, pred in zip(actual_values, predicted_values):
        sq_errors.append((act-pred)**2)
    # Calculate the mean squared error (MSE) by dividing the total squared␣
↪error by the number of values
    return sum(sq_errors)*1.0/len(sq_errors)
```

[8]:
```
# Testing the function calculating MSE

# Test 1: Simple integers, with small-ish errors
act1 = [10, 20, 30]
pre1 = [12, 18, 25]
# Calculation: (|10-12| + |20-18| + |30-25|) / 3 = (2 + 2 + 5) / 3 = 3.0
assert get_mean_absolute_error(act1, pre1) == 3.0
print(f"Test 1 small-ish errors MAE: {get_mean_absolute_error(act1, pre1)}")
# Calculation: ((10-12)^2 + (20-18)^2 + (30-25)^3) / 3 = (4 + 4 + 25) / 3 = 11.0
assert get_mean_squared_error(act1, pre1) == 11.0
print(f"Test 1 small-ish errors MSE: {get_mean_squared_error(act1, pre1)}\n")

# Test 2: Perfect match (Error should be 0)
act2 = [1.5, 2.5, 3.5]
pre2 = [1.5, 2.5, 3.5]
# Calculation: (|1.5-1.5| + |2.5-2.5| + |3.5-3.5|) / 3 = (0 + 0 + 0) / 3 = 0
assert get_mean_absolute_error(act2, pre2) == 0
print(f"Test 2 perfect match MAE: {get_mean_absolute_error(act2, pre2)}")
assert get_mean_squared_error(act2, pre2) == 0
print(f"Test 2 perfect match MSE: {get_mean_squared_error(act2, pre2)}\n")
```

```
# Test 3: Large differences
act3 = [100, 0]
pre3 = [0, 100]
# Calculation: (|100-0| + |0-100|) / 2 = (100 + 100) / 2 = 100.0
assert get_mean_absolute_error(act3, pre3) == 100
print(f"Test 3 large errors MAE: {get_mean_absolute_error(act3, pre3)}")
# Calculation: ((100-0)^2 + (0-100)^2) / 2 = (10000 + 10000) / 2 = 10000
assert get_mean_squared_error(act3, pre3) == 10000
print(f"Test 3 large errors MSE: {get_mean_squared_error(act3, pre3)}\n")

# Test 3: The "Outlier" penalty
act4 = [10, 10,10,10,10]
pre4 = [0, 10,10,10,10]
# Notice: One error is 10, many others are 0.
# Calculation: (|10-0| + |10-10| + |10-10| + |10-10| + |10-10|) / 5 = 10/5 =2
assert get_mean_absolute_error(act4, pre4) == 2
print(f"Test 4 outlier penalty MAE: {get_mean_absolute_error(act4, pre4)}")
# Calculation: (10^2 + 4\cdot 0^2) / 5 = 100 / 5 = 20
assert get_mean_squared_error(act4, pre4) == 20
print(f"Test 4 outlier penalty MSE: {get_mean_squared_error(act4, pre4)}")
```

```
Test 1 small-ish errors MAE: 3.0
Test 1 small-ish errors MSE: 11.0

Test 2 perfect match MAE: 0.0
Test 2 perfect match MSE: 0.0

Test 3 large errors MAE: 100.0
Test 3 large errors MSE: 10000.0

Test 4 outlier penalty MAE: 2.0
Test 4 outlier penalty MSE: 20.0
```

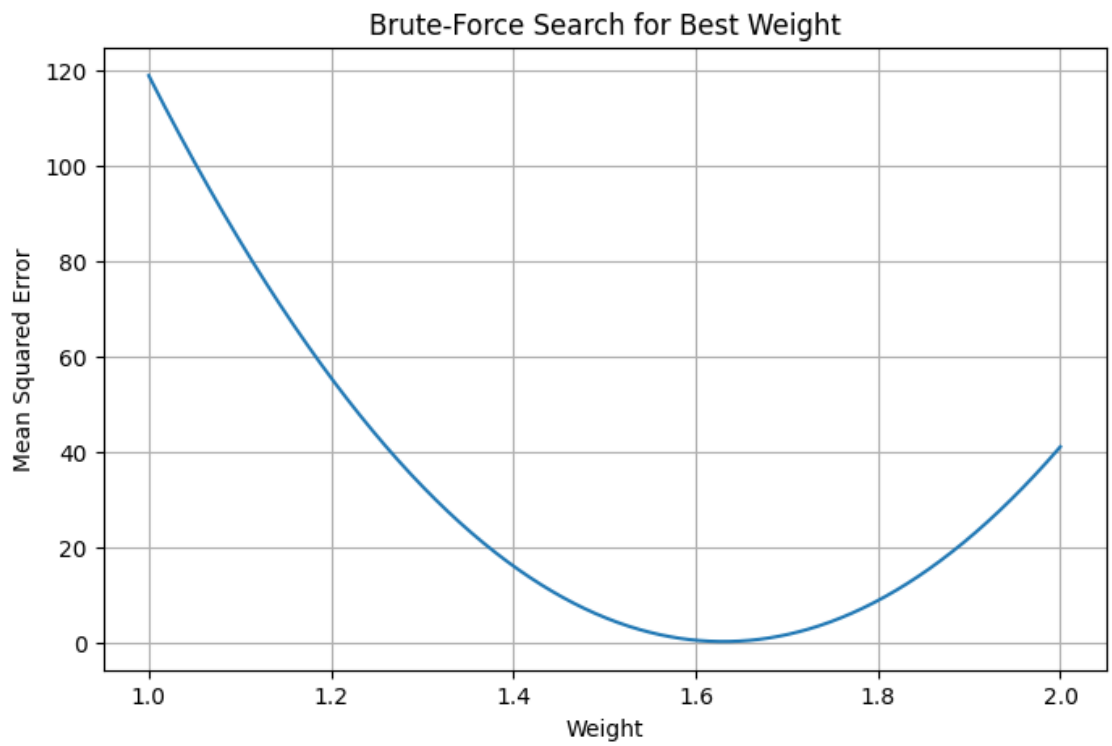
Let's run our brute force search again, but this time, we will use the MSE intead.

[9]:
```
# Brute-force search for the best weight (conversion factor)
# with bias fixed to 0, using Mean Square Error as the error metric

best_weight = None # Initialize best weight
lowest_error = float('inf') # Initialize to a very high value

weights = [] # To store weights tried
mse_list = [] # To store mean squared errors for each weight

w = 1.0
b = 0.0
step_size = 0.001
```

```
while w <= 2.0:
    # Predict output values for each data point given current model
    predicted = get_predictions_v2(miles, w, b)
    # Calculate mean squared error relative to data sets and current model
    mean_squared_error = get_mean_squared_error(km, predicted)
    # For the plot, update the weights list and the list of mean squared errors
    mse_list.append(mean_squared_error)
    weights.append(w)
    # Update best parameters so far
    if mean_squared_error < lowest_error:
        lowest_error = mean_squared_error
        best_weight = w
    w += step_size

print(f'Best weight (conversion factor): {best_weight:.4f}')
print(f'Lowest mean squared error: {lowest_error:.2f}')
```

```
Best weight (conversion factor): 1.6300
Lowest mean squared error: 0.18
```

[10]:
```
plt.figure(figsize=(8,5))
plt.plot(weights, mse_list)
plt.xlabel("Weight")
plt.ylabel("Mean Squared Error")
plt.title("Brute-Force Search for Best Weight")
plt.grid()
plt.show()
```



1.1.3 Error as a function

A function is given with a *rule* from inputs to outputs.

For a given training data set, the error e, is a function of the weight, w.

Tuning the parameters of the model means considering different values for *weight* along the x - axis. After graphing the error function, for each x - axis value, we can read from the graph the corresponding y - axis value, namely what the error is.

`Finding the model with best fit means minimizing the error.` Graphically: this means finding the value for the weight at which the graph is lowest.

Using this insight, let's explore a different approach to finding the best parameters for our model, instead of brute force search.

Instead of searching through *all* possible weight values, we can strategically explore the search space and choose to update our current estimate for the weight value based on how we estimate that the error will change.

To formalize this, we use **derivatives**. 3Blue1Brown Video Link for Derivative

Intuition: to estimate how the error changes based on changing the weight value, we consider the rate of change of the function close to the current point.

In formulas, if we write the error as a function of the weight, $e = f(w)$. Whether or not we have a nice formula for this function, we can observe some of its properties.

The ratio of how the error changes relative to a change ϵ in weight is

$$\frac{\tilde{e} - e}{\tilde{w} - w} = \frac{f(w + \epsilon) - f(w)}{\epsilon}$$

This is a **finite difference approximation** to the **derivative**. Formally, we can express the derivative as the *limit* of the finite difference approximation:

$$\frac{de}{dw} = \lim_{\epsilon \to 0} \frac{f(w + \epsilon) - f(w)}{\epsilon}$$

where we can informally think of dw as a "small change in the weight" which results in dw, the "change in error".

[11]:
```
# Complete the function that approximates the derivative of error with respect␣
↪to weight (w) using finite differences.

def get_derivative_error_wrt_weight(error_function, input_values,␣
↪actual_values, w, b, dw=1e-6):
    """
    Calculates the derivative of the error with respect to weight (w) using the␣
↪limit definition.

    error_function: function to compute error
    input_values: list of input values
    actual_values: list of actual output values
    w: current weight
    b: current bias

    dw: small value for finite difference approximation

    Returns:
        derivative (float): Approximate derivative of error with respect to w
    """
    # Get predictions at current w
    predicted_w = get_predictions_v2(input_values, w, b)

    # Compute error at w
    error_w = error_function(actual_values, predicted_w)

    # Get predictions at w + dw
    predicted_w_dw = get_predictions_v2(input_values, w + dw, b)

    # Compute error at w + dw
    error_w_dw = error_function(actual_values, predicted_w_dw)

    # Approximate and return the derivative value,
    # using the finite difference expression
    # YOUR CODE HERE
```

```
    derivative = (error_w_dw-error_w)/dw
    return derivative
```

[12]:
```
# TEST CASE FOR DERIVATIVE FUNCTION (get_derivative_error_wrt_weight)

# Approximate the derivative of the error with respect to weight at w=1

derivative = get_derivative_error_wrt_weight(
    get_mean_absolute_error, miles, km, 1, b
)

# Should print -13.8000
print(f"Derivative of error with respect to weight at w={1:.2f} is␣
 ↪approximately {derivative:.4f}")
```

```
Derivative of error with respect to weight at w=1.00 is approximately -13.8000
```

1.1.4 Big picture

Once we express error as a function of the model parameters, we can use the derivative of this function to guide our search for the best parameters, those that minimize this function. We want to use the derivative to approximate the rate of the change of error as we tune the weight parameter. We can visualize that approximation by drawing the **tangent line** to the curve: a line that intersects with the graph and whose slope is the rate of change of the graph.

This process depends on what function we use the calculate the error. While Mean Absolute Error (MAE) is easy to interpret, its mathematical properties make minimizing (optimizing) it using derivative based methods challenging. Instead, Mean Squared Error (MSE) is commonly used.

In calculus classes, you'll talk about continuity, differentiability, and smoothness of functions. It turns out that MAE may not be differentiable at zero, which is a problem for this optimization approach.

Run the cell below and use the slider to see how changing the weight affects the error and the tangent line at that point on the curve.

[13]:
```
import numpy as np
from ipywidgets import interact, FloatSlider

def plot_tangent(weight):
    # Find the closest index in weights to the selected weight
    idx = np.abs(np.array(weights) - weight).argmin()
    x0 = weights[idx]
    y0 = mse_list[idx]

    # Use the derivative function to get the slope at x0
    slope = get_derivative_error_wrt_weight(
        get_mean_squared_error, miles, km, x0, 0.0
    )
```

```
    # Tangent line: y = slope * (x - x0) + y0, but keep it short
    x_tangent = np.linspace(x0 - 0.05, x0 + 0.05, 20)
    y_tangent = slope * (x_tangent - x0) + y0

    plt.figure(figsize=(8,5))
    plt.plot(weights, mse_list, label="Error Curve")
    plt.plot(x_tangent, y_tangent, 'r--', label=f"Tangent at w={x0:.3f}")
    plt.scatter([x0], [y0], color='red', zorder=5)
    plt.xlabel("Weight")
    plt.ylabel("Mean Squared Error")
    plt.title("Interactive Tangent to Error Curve")
    plt.legend()
    plt.grid()
    plt.show()

interact(plot_tangent, weight=FloatSlider(min=min(weights), max=max(weights) -␣
↪0.01, step=0.01, value=1, description='Weight'))
```

```
interactive(children=(FloatSlider(value=1.0, description='Weight', max=1.
↪9899999999998899, min=1.0, step=0.01)…
```

[13]:
```
<function __main__.plot_tangent(weight)>
```

What to Notice in the Tangent Line Graph

- As you move the weight slider, notice how the red dashed tangent line changes.
- The tangent line always touches the error curve at exactly one point (the current weight).
- **Pay attention to the angle (slope) of the tangent line:**
  - Where the error curve is steep, the tangent line is also steep (more vertical).
  - Where the error curve is flat (near the minimum), the tangent line is almost horizontal.
- **The length of the tangent line segment (visually)** appears to change:
  - This is because the **vertical distance** covered by the line depends on the slope.
  - The x-range is always the same, but a steeper slope makes the line look longer up and down.
- **Try to find the minimum:**
  - At the lowest point of the error curve, the tangent line is flat (slope = 0).
  - This is where the error is minimized and the model is best.

  Key idea:
  The tangent line shows how the error would change if you increased or decreased the weight a little bit: if its slope is negative, then increasing weight would decrease error; if its slope is positive, then decreasing weight would decrease error. The steeper the tangent, the faster the error is changing at that point.

Quick check: before, we estimated that the derivative of MSE at $w = 1.0$ was approximately -13.8. Does this mean that increasing w will `worsen` or `improve` the performance?

YOUR ANSWER HERE

Quick check: If there's some weight value w_0 where adding a little bit to the weight gives an error value that is `greater` than the old error value, what is the sign of this fraction?

$$\frac{f(w_0 + \epsilon) - f(w_0)}{\epsilon}$$

YOUR ANSWER HERE

[14]:
```
## For comparison: we can look at the tangent to the MAE error curve and notice␣
↪that it has some problematic properties at the minimum of the error curve.

import numpy as np
from ipywidgets import interact, FloatSlider

def plot_tangent(weight):
    # Find the closest index in weights to the selected weight
    idx = np.abs(np.array(weights) - weight).argmin()
    x0 = weights[idx]
    y0 = mae_list_original[idx]

    # Use the derivative function to get the slope at x0
    slope = get_derivative_error_wrt_weight(
        get_mean_absolute_error, miles, km, x0, 0.0
    )

    # Tangent line: y = slope * (x - x0) + y0, but keep it short
    x_tangent = np.linspace(x0 - 0.05, x0 + 0.05, 20)
    y_tangent = slope * (x_tangent - x0) + y0

    plt.figure(figsize=(8,5))
    plt.plot(weights, mae_list_original, label="Error Curve")
    plt.plot(x_tangent, y_tangent, 'r--', label=f"Tangent at w={x0:.3f}")
    plt.scatter([x0], [y0], color='red', zorder=5)
    plt.xlabel("Weight")
    plt.ylabel("Mean Absolute Error")
    plt.title("Interactive Tangent to Error Curve")
    plt.legend()
    plt.grid()
    plt.show()

interact(plot_tangent, weight=FloatSlider(min=min(weights), max=max(weights) -␣
↪0.01, step=0.01, value=1, description='Weight'))
```

```
interactive(children=(FloatSlider(value=1.0, description='Weight', max=1.
↪9899999999998899, min=1.0, step=0.01)…
```

[14]: <function __main__.plot_tangent(weight)>

1.1.5 Gradient Descent

The derivative of the **error with respect to a parameter** tells us two things.

1. Direction (sign)

- If the derivative is **positive**, increasing the parameter increases the error.
  To reduce error, we should **decrease** the parameter.
- If the derivative is **negative**, increasing the parameter decreases the error.
  To reduce error, we should **increase** the parameter.

Key idea: to reduce error, move the parameter in the **opposite direction of the derivative**.

2. Amount (magnitude)

- A **large** derivative means the error is changing quickly **at the current parameter value**.
- A **small** derivative means the error is changing slowly.

The derivative tells us how steep the error surface is, but not how far we should move.

Putting this together, we update the parameter by moving **opposite the derivative**, scaled by a small constant that controls how big each step is.

This idea is exactly what **`gradient descent`** formalizes.

Adding the learning rate The derivative gives us the direction we should move and how steep the error surface is locally. However, we usually do **not** want to move the full amount suggested by the derivative.

If we always took the full step, updates could be too large and overshoot the minimum.

To control this, we introduce a **learning rate**.

The learning rate is a small positive number that *scales the update*. It lets us move in the right direction, but in *small, controlled steps*.

Putting this together, we update the parameter by: - Moving in the **opposite direction of the derivative**, and - Scaling that movement by the **learning rate**.

This gives the `update rule`:

$$\text{new_parameter} = \text{old_parameter} - (\text{learning rate}) \cdot (\text{derivative})$$

With this rule: - The **derivative** determines the direction of the update. - The **learning rate** determines how big each step is.

Repeating this update over many steps is what we call **gradient descent**.

You can see this process visually as moving downhill on the error curve, step by step.

Implementing Gradient Descent Fill in the missing code below to complete a simple gradient descent algorithm for finding the best weight. You will:

- Use the current weight to make predictions.
- Compute the mean squared error for those predictions.
- Calculate the (approximate) derivative of the error with respect to the weight.
- Update the weight using the gradient descent rule.

Goal:
Your code should print: - `Best weight found by gradient descent: 1.6303` - `Mean squared error at best weight: 0.1823`

Replace each `# YOUR CODE HERE` with the correct line of code.

[15]:
```
# Gradient Descent - Complete the code below
# It should print:
# Best weight found by gradient descent: 1.6303
# Mean squared error at best weight: 0.1823

# Step 0: Initialize parameters
w_gd = 10.0
b_gd = 0.0

# Set the learning rate
learning_rate = 0.001

# Set the number of gradient descent steps
num_steps = 1000

# Lists to track how weight and error change over time
weights_gd = []
errors_gd = []

for step in range(num_steps):

    # YOUR CODE HERE
    predicted = get_predictions_v2(miles, w, b)
    # Calculate mean squared error relative to data sets and current model
    mean_squared_error = get_mean_squared_error(km, predicted)
    # For the plot, update the weights list and the list of mean squared errors
    mse_list.append(mean_squared_error)
    # DELETE FROM THIS FOR RELEASED VERSION
    # Use the current weight to make predictions
    predicted = get_predictions_v2(miles, w_gd, b_gd)
    # Compute the mean squared error of the predictions
    error = get_mean_squared_error(km,predicted)
    # Track the current weight and error
    weights_gd.append(w_gd)
    errors_gd.append(error)
```

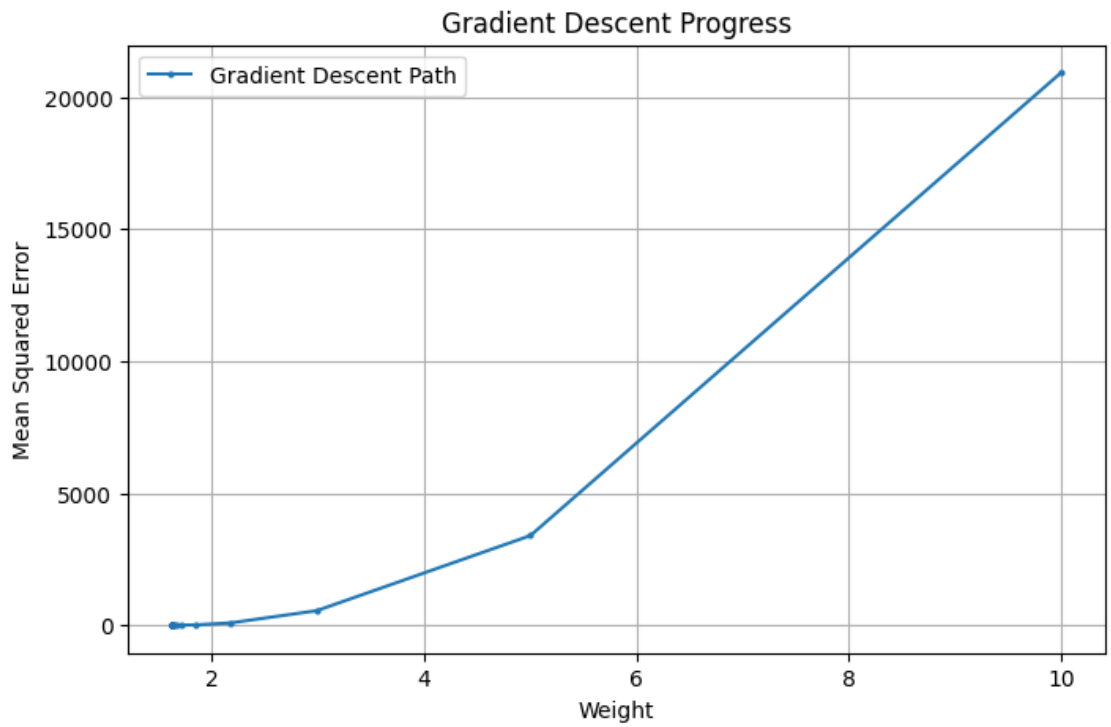
```
    # Compute the derivative of the error with respect to the weight
    grad =␣
↪get_derivative_error_wrt_weight(get_mean_squared_error,miles,km,w_gd,b_gd,dw=1e-6)
    # Update the weight using gradient descent
    w_gd = w_gd-learning_rate*grad
    # DELETE UNTIL THIS FOR RELEASED VERSION

print(f"Best weight found by gradient descent: {w_gd:.4f}")
print(f"Mean squared error at best weight: {errors_gd[-1]:.4f}")
```

```
Best weight found by gradient descent: 1.6303
Mean squared error at best weight: 0.1823
```

Plotting the sequence of weight values in the gradient descent approach highlights its advantage over brute force search: we get to make much quicker progress when we're far away from the goal, and then hone in as we get close.

Brute-force search uses a fixed step-size to update the weight, but gradient descent takes bigger steps when the slope is large and smaller steps when the slope is small.

[16]:
```
plt.figure(figsize=(8,5))
plt.plot(weights_gd, errors_gd, marker='o', markersize=2, label='Gradient␣
↪Descent Path')
plt.xlabel("Weight")
plt.ylabel("Mean Squared Error")
plt.title("Gradient Descent Progress")
plt.grid()
plt.legend()
plt.show()
```



1.2 Models with more than one parameter: the temperature example

So far, we've looked at gradient descent for a model with a single parameter (weight). We can also take derivatives with respect to more than one parameter.

Let's think back to our temperature conversion model: converting degrees Fahrenheit to degrees Celsius.

[17]:
```
# Data for Fahrenheit and Celsius
fahrenheit = [32, 50, 68, 86, 104] # x-axis values (INPUT DATA)
celsius = [0, 10, 20, 30, 40] # y-axis values (OUTPUT DATA)


best_weight = None
best_bias = None
lowest_error = float('inf')

weights = []
weight_step_size = 0.01

biases = []
bias_step_size = 0.5

errors_for_plot_mae = []
```

```
errors_for_plot_mse = []

weight = 0.4
while weight <= 0.7:
    bias = -30.0
    while bias <= -10.0:
        mean_absolute_error = get_mean_absolute_error(
            celsius,
            get_predictions_v2(fahrenheit, weight, bias)
        )
        mean_squared_error = get_mean_squared_error(
            celsius,
            get_predictions_v2(fahrenheit, weight, bias)
        )

        weights.append(weight)
        biases.append(bias)
        errors_for_plot_mae.append(mean_absolute_error)
        errors_for_plot_mse.append(mean_squared_error)


        if mean_squared_error < lowest_error:
            lowest_error = mean_squared_error
            best_weight = weight
            best_bias = bias

        bias += bias_step_size
    weight += weight_step_size

print(f"Best weight: {best_weight:.4f}")
print(f"Best bias: {best_bias:.2f}")
print(f"Lowest total error: {lowest_error:.2f}")
```

```
Best weight: 0.5600
Best bias: -18.00
Lowest total error: 0.02
```

Our visualizations of error will be more complicated now.

Run the cell below to see plots of the error surface in 3D, using mean absolute error (MAE) and mean squared error (MSE).

[18]:
```
# May need to run pip install plotly
import plotly.graph_objects as go

# MAE plot
fig_mae = go.Figure(data=[go.Scatter3d(
    x=weights,
```

```
    y=biases,
    z=errors_for_plot_mae,
    mode='markers',
    marker=dict(
        size=5,
        color=errors_for_plot_mae,
        colorscale='Viridis',
        colorbar=dict(title='MAE')
    )
)])
fig_mae.add_trace(go.Scatter3d(

    x=[best_weight],
    y=[best_bias],
    z=[min(errors_for_plot_mae)],
    mode='markers',
    marker=dict(size=10, color='red'),
    name='Best Parameters'
))
fig_mae.update_layout(
    scene=dict(
        xaxis_title='Weight',
        yaxis_title='Bias',
        zaxis_title='MAE'
    ),
    title='MAE Surface: Weight vs Bias',

    width=1000,  # Increase width
    height=800   # Increase height
)
fig_mae.show()

# MSE plot
fig_mse = go.Figure(data=[go.Scatter3d(
    x=weights,
    y=biases,
    z=errors_for_plot_mse,
    mode='markers',
    marker=dict(
        size=5,
        color=errors_for_plot_mse,
        colorscale='Viridis',
        colorbar=dict(title='MSE')
    )
)])
fig_mse.add_trace(go.Scatter3d(
    x=[best_weight],
```

```
    y=[best_bias],
    z=[min(errors_for_plot_mse)],
    mode='markers',
    marker=dict(size=10, color='red'),
    name='Best Parameters'
))
fig_mse.update_layout(
    scene=dict(
        xaxis_title='Weight',
        yaxis_title='Bias',
        zaxis_title='MSE'
    ),
    title='MSE Surface: Weight vs Bias',
    width=1000,  # Increase width
    height=800   # Increase height
)
fig_mse.show()
```

1.2.1 Calculating the Gradient with Respect to Both w and b

When our model is

$$y = wx + b$$

the error depends on both w (weight) and b (bias). To take into account both parameters when finding the values of parameters that give least error, we consider how the error changes with respect to each parameter in turn. This is called computing the **partial derivatives** (components of gradient) of the error with respect to each parameter:

- $\frac{\partial e}{\partial w}$: How the error changes as w changes, keeping b fixed.
- $\frac{\partial e}{\partial b}$: How the error changes as b changes, keeping w fixed.

We can estimate these using the finite difference approximation:

$$\frac{\partial e}{\partial w} \approx \frac{e(w+\epsilon, b) - e(w, b)}{\epsilon}$$

$$\frac{\partial e}{\partial b} \approx \frac{e(w, b+\epsilon) - e(w, b)}{\epsilon}$$

Where $e(w, b)$ is the error function, and ϵ is a small number. These gradients can then be used to update both w and b in gradient descent.

Fill in the missing code below to complete the function that approximates the partial derivatives (gradients) of the error with respect to both the weight (w) and the bias (b) using finite differences.

You will: - Compute predictions and error at the current values of model parameters (w, b). - Compute predictions and error at $(w+\epsilon, b)$ and $(w, b+\epsilon)$. - Approximate the partial derivatives with respect to w and b. - Return both gradients.

Goal:
Replace each `# YOUR CODE HERE` with the correct line of code so the function returns the correct gradients for w and b.

[19]:
```
def get_gradients_w_b(error_function, input_values, actual_values, w, b,␣
↪epsilon=1e-6):
    """
    Approximates the partial derivatives of the error with respect to w and b
    using finite differences.

    Returns:
        (grad_w, grad_b)
    """

    # Step 1: Compute predictions using the current parameters (w, b)
    predicted = get_predictions_v2(input_values,w,b)
    # YOUR CODE HERE

    # Step 2: Compute the error at (w, b)
    error = error_function(actual_values,predicted)
    # YOUR CODE HERE

    # --- Partial derivative with respect to w ---

    # Step 3: Compute predictions at (w + epsilon, b)
    predicted_w = get_predictions_v2(input_values, w+epsilon, b)
    # YOUR CODE HERE

    # Step 4: Compute the error at (w + epsilon, b)
    error_w = error_function(actual_values,predicted_w)
    # YOUR CODE HERE

    # Step 5: Approximate the partial derivative with respect to w
    grad_w = (error_w - error)/epsilon
    # YOUR CODE HERE

    # --- Partial derivative with respect to b ---

    # Step 6: Compute predictions at (w, b + epsilon)
    predicted_b = get_predictions_v2(input_values,w,b+epsilon)
    # YOUR CODE HERE

    # Step 7: Compute the error at (w, b + epsilon)
    error_b = error_function(actual_values,predicted_b)
    # YOUR CODE HERE

    # Step 8: Approximate the partial derivative with respect to b
```

```
    grad_b = (error_b-error)/epsilon
    # YOUR CODE HERE

    return grad_w, grad_b
```

[20]:
```
# TEST CASE FOR GRADIENT FUNCTION (get_gradients_w_b)
#
# Approximate the partial derivatives of the MSE with respect to
# weight and bias at w=0.55 and b=-17

w_gradient, b_gradient = get_gradients_w_b(
    get_mean_squared_error, fahrenheit, celsius, 0.55, -17
)

# Should print 47.20527200133274 0.8000010005004832
print(w_gradient, b_gradient)
```

```
47.20527200136049 0.8000010005559943
```

Putting this together, gradient descent for this two parameter model updates the parameters each step by: - Moving in the **opposite direction of the derivative**, and - Scaling that movement by the **learning rate**.

Repeating this update over many steps is what we call **gradient descent**. You can see this process visually as moving downhill on the **error surface**, step by step.

[21]:
```
# Gradient Descent for Fahrenheit to Celsius conversion using both w and b

# Initialize parameters
w_fc = 1
b_fc = -60

# Set the learning rate
# TRY DIFFERENT LEARNING RATES TO SEE HOW MSE changes with each step
# For example, try 0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001
learning_rate_fc = 0.0001

# Set the number of gradient descent steps
num_steps_fc = 250000

grad_weights_fc = []
grad_biases_fc = []
grad_errors_fc = []

for step in range(num_steps_fc):
    predicted_fc = get_predictions_v2(fahrenheit, w_fc, b_fc)
    error_fc = get_mean_squared_error(celsius, predicted_fc)
    grad_weights_fc.append(w_fc)
    grad_biases_fc.append(b_fc)
```

```
    grad_errors_fc.append(error_fc)
    grad_w_fc, grad_b_fc = get_gradients_w_b(get_mean_squared_error,␣
 ↪fahrenheit, celsius, w_fc, b_fc)
    w_fc = w_fc - learning_rate_fc * grad_w_fc
    b_fc = b_fc - learning_rate_fc * grad_b_fc
    if step % 25000 == 0:
        print(f"Step {step}: w = {w_fc:.4f}, b = {b_fc:.4f}, error = {error_fc:.
 ↪4f}")

print(f"Best weight (w) found: {w_fc:.4f}")
print(f"Best bias (b) found: {b_fc:.4f}")
print(f"Mean squared error at best parameters: {grad_errors_fc[-1]:.4f}")

# Plot the Gradient Descent Path
import plotly.graph_objects as go

fig_mse = go.Figure(data=[go.Scatter3d(
    x=grad_weights_fc,
    y=grad_biases_fc,
    z=grad_errors_fc,
    mode='markers',
    marker=dict(
        size=5,
        color=grad_errors_fc,
        colorscale='Viridis',
        colorbar=dict(title='MSE')
    )
)])
fig_mse.add_trace(go.Scatter3d(
    x=[w_fc],
    y=[b_fc],
    z=[min(grad_errors_fc)],
    mode='markers',
    marker=dict(size=10, color='red'),
    name='Best Parameters'
))
fig_mse.update_layout(
    scene=dict(
        xaxis_title='Weight',
        yaxis_title='Bias',
        zaxis_title='MSE'
    ),
    title='MSE Surface: Weight vs Bias',
    width=1000,  # Increase width
    height=800   # Increase height
)
fig_mse.show()
```

```
Step 0: w = 1.1056, b = -59.9976, error = 272.0000
Step 25000: w = 0.8501, b = -40.6155, error = 64.1100
Step 50000: w = 0.7149, b = -30.1312, error = 18.7583
Step 75000: w = 0.6417, b = -24.4599, error = 5.4885
Step 100000: w = 0.6022, b = -21.3922, error = 1.6058
Step 125000: w = 0.5808, b = -19.7328, error = 0.4698
Step 150000: w = 0.5692, b = -18.8352, error = 0.1374
Step 175000: w = 0.5629, b = -18.3496, error = 0.0402
Step 200000: w = 0.5595, b = -18.0870, error = 0.0118
Step 225000: w = 0.5577, b = -17.9449, error = 0.0034
Best weight (w) found: 0.5567
Best bias (b) found: -17.8681
Mean squared error at best parameters: 0.0010
```

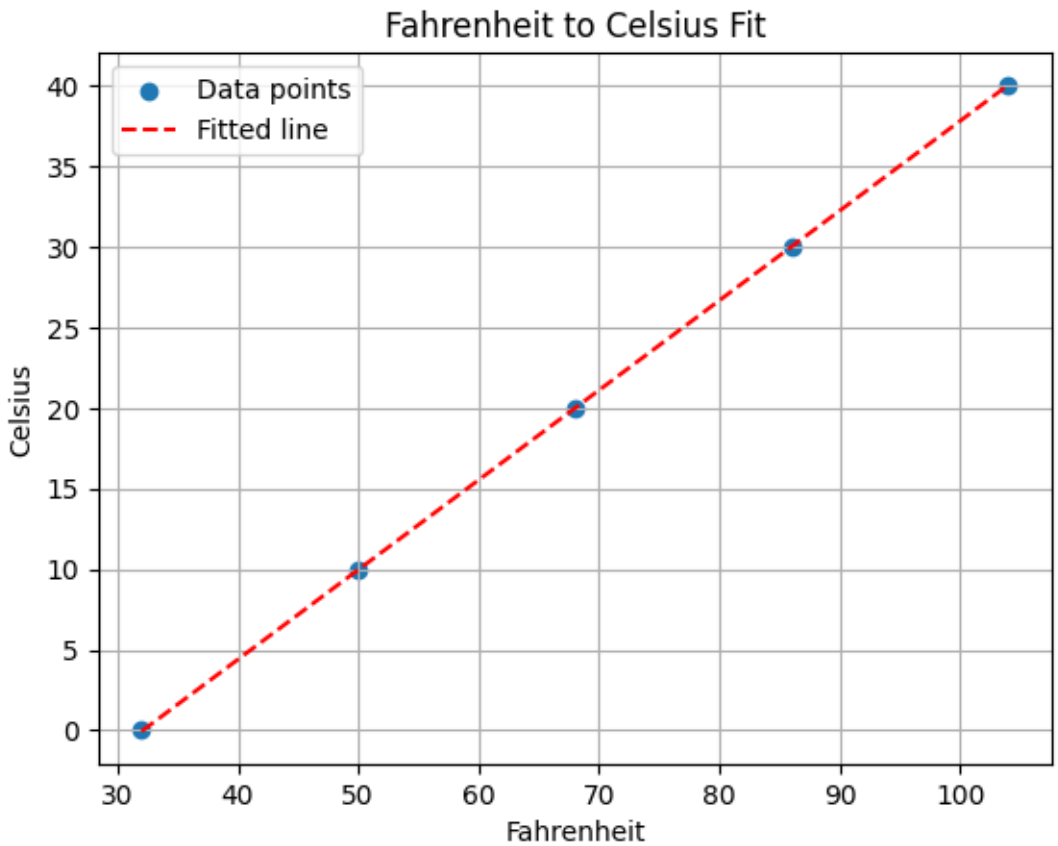
1.2.2 What just happened?

Earlier, we said: - The derivative gives the **direction**. - The learning rate controls **how big the step is**. - The derivative is only a **local** signal.

In some runs, the learning rate was too large. That caused each update to be much larger than the local information from the derivative could support. Instead of moving downhill, the algorithm jumped past the minimum and off the error surface. Nothing about the direction was wrong. The steps were simply too big.

In some runs, the learning rate was too small. That caused each update to nudge a little downhill, but we ran out of steps before getting close to the minimum.

[22]:
```
# Plot the data points and the fitted line
import matplotlib.pyplot as plt

# Original data
plt.scatter(fahrenheit, celsius, label='Data points')

x_line = [min(fahrenheit), max(fahrenheit)]
y_line = [w_fc * x + b_fc for x in x_line]
plt.plot(x_line, y_line, color='red', linestyle='--',  label='Fitted line')

plt.xlabel('Fahrenheit')
plt.ylabel('Celsius')
plt.title('Fahrenheit to Celsius Fit')
plt.grid()
plt.legend()
plt.show()
```



1.2.3 What do we do with the model?

Now that we've fit a line, we can use it to make predictions for new, unseen data.

For example, if we have a temperature in Fahrenheit, we can use the model to estimate the temperature in Celsius.

[23]:
```
# Predict Celsius for new Fahrenheit values using the learned model
new_fahrenheit = [40, 77, 100]
# Get predictions for the new Fahrenheit values
predicted_celsius = get_predictions_v2(new_fahrenheit, w_fc, b_fc)

for f, c in zip(new_fahrenheit, predicted_celsius):
    print(f"Fahrenheit: {f} -> Predicted Celsius: {c:.2f}")
```

```
Fahrenheit: 40 -> Predicted Celsius: 4.40
Fahrenheit: 77 -> Predicted Celsius: 25.00
Fahrenheit: 100 -> Predicted Celsius: 37.80
```

1.2.4 More parameters: From a Line to Multiple Variables

We've been developing a method for modeling the relationship between input variables (**features**) and an output variable (**target**) by fitting a straight line.

The Equation of a Line (One Variable) For a single input variable x, the equation of a line is:

$$y = w \cdot x + b$$

- y = predicted value (output)
- x = input value (feature)
- w = weight (slope of the line)
- b = bias (intercept)

This is the basic form of **simple linear regression**. We have already seen this!

Generalizing to Multiple Input Variables When there are multiple input variables (features), we use **multiple linear regression**. The equation becomes:

$$y = w_1 x_1 + w_2 x_2 + ... + w_n x_n + b$$

So, the prediction is a **weighted sum** of all the input features plus a bias:

$$y = \sum_{i=1}^{n} w_i x_i + b$$

The model learns the best weights $(w_1, w_2, ..., w_n)$ and bias (b) to fit the data.

What is a Hyperplane? A **hyperplane** is a generalization of the concept of a plane to higher dimensions. In two dimensions, this is a line; in three dimensions, it is a flat surface or a plane. When we move to four or more dimensions, we call this flat, infinitely extending surface a hyperplane.

- In **2D**, a hyperplane is a line.
- In **3D**, a hyperplane is a plane.
- In **nD** (where n > 3), a hyperplane is an (n-1)-dimensional subspace.

Take-away:
- With one input variable, linear regression fits a line.
- With many input variables, it fits a *hyperplane* in higher dimensions.
- The principle is the same: predict y as a weighted sum of the inputs plus a bias.

Example of model with multiple input variables Consider the problem of computing the total amount of money given a specific number of *dimes* and specific number of *nickels*.

We want to predict the total value using a linear model that takes the number of nickels and dimes as input features. This is an example of a linear regression problem with two variables.

If we think about an input variable x_1 representing the number of nickels and an input variable x_2 representing the number of dimes, we can use the following equation to compute the total output, y:

$$y = 5x_1 + 10x_2$$

Quick check: why is the bias set to 0?

We can illustrate this relationship with a table of values, as well as with a 3D plot.

[24]:
```
# Illustrating this relationship

import matplotlib.pyplot as plt
import plotly.graph_objs as go

dimes_range = range(11)
nickels_range = range(11)
grid = []

for n in nickels_range:
    row = []
    for d in dimes_range:
        total = 5 * n + 10 * d
        row.append(total)
    grid.append(row)

fig, ax = plt.subplots(figsize=(7,6))
ax.set_xticks(range(len(dimes_range)))
ax.set_yticks(range(len(nickels_range)))
ax.set_xticklabels(dimes_range)
ax.set_yticklabels(nickels_range)
ax.set_xlabel('Number of Dimes')
ax.set_ylabel('Number of Nickels')
ax.set_title('Total Value for Dimes and Nickels')

# Draw grid lines
for i in range(len(nickels_range)+1):
    ax.axhline(i-0.5, color='gray', linewidth=0.5)
for j in range(len(dimes_range)+1):
    ax.axvline(j-0.5, color='gray', linewidth=0.5)

# Fill in numbers
for i, n in enumerate(nickels_range):
    for j, d in enumerate(dimes_range):
        ax.text(j, i, f"{grid[i][j]}", va='center', ha='center', fontsize=12)

ax.set_xlim(-0.5, len(dimes_range)-0.5)
ax.set_ylim(-0.5, len(nickels_range)-0.5)
ax.invert_yaxis()
```

```
plt.gca().invert_yaxis()
plt.show()


# prepare data for 3D plot
all_x1 = []
all_x2 = []
all_y = []
for i, n in enumerate(nickels_range):
    for j, d in enumerate(dimes_range):
        all_x1.append(n)
        all_x2.append(d)
        all_y.append(grid[i][j])
import plotly.graph_objs as go

# Create a 3D scatter plot using plotly
scatter = go.Scatter3d(
    x=all_x1,
    y=all_x2,
    z=all_y,
    mode='markers',
    marker=dict(size=5, color=all_y, colorscale='Viridis'),
    name='Total Value'
)

layout = go.Layout(
    scene=dict(
        xaxis_title='# Nickels',
        yaxis_title='# Dimes',
        zaxis_title='Total'
    ),
    title='3D Plot: Total Value for Dimes and Nickels'
)
layout.update(width=800, height=600)
fig = go.Figure(data=[scatter], layout=layout, )
fig.show()
```

Total Value for Dimes and Nickels

Number of Nickels \ Number of Dimes	0	1	2	3	4	5	6	7	8	9	10
10	50	60	70	80	90	100	110	120	130	140	150
9	45	55	65	75	85	95	105	115	125	135	145
8	40	50	60	70	80	90	100	110	120	130	140
7	35	45	55	65	75	85	95	105	115	125	135
6	30	40	50	60	70	80	90	100	110	120	130
5	25	35	45	55	65	75	85	95	105	115	125
4	20	30	40	50	60	70	80	90	100	110	120
3	15	25	35	45	55	65	75	85	95	105	115
2	10	20	30	40	50	60	70	80	90	100	110
1	5	15	25	35	45	55	65	75	85	95	105
0	0	10	20	30	40	50	60	70	80	90	100

1.2.5 Fitting a Plane to Unknown Coins

Now let's try to fit a plane through some unknown coins!

Run the next two cells and use the sliders to adjust the weights `w1` and `w2` for each coin. `x1` and `x2` denote the numbers of each type of coin.

Hint: The plane should perfectly fit the data points—when you find the correct values, the plane will pass through all the points.

[25]:
```
# Unknown Coins!

# Total (y), Num coins 1 (x1), Num coins 2 (x2)
y =  [44, 23, 38, 43, 15, 18, 17, 30, 28, 21]
x1 = [ 8,  5, 10,  9,  1,  6,  5,  4,  6,  3]
x2 = [10,  4,  4,  8,  6,  0,  1,  9,  5,  6]
```

[26]:
```
# from ipywidgets import interact, FloatSlider
# import numpy as np
# import plotly.graph_objs as go

def plot_plane_fit(w1=5.0, w2=10.0):
    # Scatter plot of the data points
    scatter = go.Scatter3d(
        x=x1, y=x2, z=y,
        mode='markers',
        marker=dict(size=8, color='red'),
        name='Data points'
    )

    # Create a meshgrid for the plane
    x1_grid = np.linspace(min(x1), max(x1), 10)
    x2_grid = np.linspace(min(x2), max(x2), 10)
    X1, X2 = np.meshgrid(x1_grid, x2_grid)
    Y = w1 * X1 + w2 * X2

    plane = go.Surface(
        x=X1, y=X2, z=Y,
        opacity=0.5,
        # colorscale='Blues',
        showscale=False,
        name='Fit plane'
    )

    layout = go.Layout(
        scene=dict(
            xaxis_title='x1 (nickels)',
            yaxis_title='x2 (dimes)',
            zaxis_title='y (total value)'
        ),
        title=f'Fit: y = {w1:.0f}*x1 + {w2:.0f}*x2',
        margin=dict(l=0, r=0, b=0, t=30)
    )

    fig = go.Figure(data=[scatter, plane], layout=layout)
    display(fig)
    return None

interact(
    plot_plane_fit,
    w1=FloatSlider(value=1, min=0, max=10, step=1, description='w1'),
    w2=FloatSlider(value=1, min=0, max=10, step=1, description='w2')
)
```

```
interactive(children=(FloatSlider(value=1.0, description='w1', max=10.0, step=1.
↪0), FloatSlider(value=1.0, des…
```

[26]:
```
<function __main__.plot_plane_fit(w1=5.0, w2=10.0)>
```

[27]:
```python
# Gradient Descent for unknown coin value with two input features

# Format the input data into list that matches expected shape
X = [[x1[i],x2[i]] for i in range(len(x1))]

# Need a new get_predictions function
def get_predictions_2d(X, w, b):
    '''
    X: list of 2d lists
    w: list of two weights
    b: bias

    Calculates the predicted output values using the weights and bias
    '''
    return [input[0]*w[0] + input[1]*w[1] + b for input in X]

# Need a new get_gradients function
def get_gradients_2d_w(error_function, input_values, actual_values, w, b,␣
↪epsilon=1e-6):
    '''
    Approximates the partial derivatives of the error with respect to weights␣
↪using finite differences.

    Returns:
    (grad_w1, grad_w2)
    '''
    predicted = get_predictions_2d(input_values, w,b)
    error=error_function(actual_values,predicted)
    predicted_w1 = get_predictions_2d(input_values,[w[0]+epsilon,w[1]], b)
    error_w1 = error_function(actual_values,predicted_w1)
    grad_w1 = (error_w1 - error)/epsilon
    predicted_w2 = get_predictions_2d(input_values,[w[0],w[1]+epsilon], b)
    error_w2 = error_function(actual_values,predicted_w2)
    grad_w2 = (error_w2 - error)/epsilon
    return grad_w1,grad_w2

# Initialize parameters
w = [5,5]
b = 0

# Set the learning rate
# TRY DIFFERENT LEARNING RATES TO SEE HOW MSE changes with each step
```

```python
# For example, try 0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001
learning_rate = 0.01

# Set the number of gradient descent steps
num_steps = 250000

# Note: bias will be fixed to 0 because of the problem context

grad_w1 = []
grad_w2 = []
grad_errors = []

for step in range(num_steps):
    predicted = get_predictions_2d(X,w,b)
    error = get_mean_squared_error(y,predicted)
    grad_w1.append(w[0])
    grad_w2.append(w[1])
    grad_errors.append(error)
    dw1,dw2 = get_gradients_2d_w(get_mean_squared_error, X, y, w,b)
    w[0] = w[0] - learning_rate * dw1
    w[1] = w[1] - learning_rate * dw2
    if step % 25000 == 0:
        print(f"Step {step}: w1 = {w[0]:.4f}, w2 = {w[1]:.4f}, b = {b}, error =␣
 ↪{error:.4f}")

print(f"Best weights ([w1,w2]) found: {w}")
print(f"Best bias (b) found: {b:.4f}")
print(f"Mean squared error at best parameters: {grad_errors[-1]:.4f}")

# Plot the Gradient Descent Path
import plotly.graph_objects as go

fig_mse = go.Figure(data=[go.Scatter3d(
    x=grad_w1,
    y=grad_w2,
    z=grad_errors,
    mode='markers',
    marker=dict(
        size=5,
        color=grad_errors,
        colorscale='Viridis',
        colorbar=dict(title='MSE')
    )
)])
fig_mse.add_trace(go.Scatter3d(
    x=[grad_w1[-1]],
    y=[grad_w2[-1]],
```

```
    z=[grad_errors[-1]],
    mode='markers',
    marker=dict(size=10, color='red'),
    name='Best Parameters'
))
fig_mse.update_layout(
    scene=dict(
        xaxis_title='w1',
        yaxis_title='w2',
        zaxis_title='MSE'
    ),
    title='MSE Surface: w1, w2',
    width=1000,  # Increase width
    height=800   # Increase height
)
fig_mse.show()
```

```
Step 0: w1 = 1.5860, w2 = 1.5220, b = 0, error = 863.1000
Step 25000: w1 = 3.0000, w2 = 2.0000, b = 0, error = 0.0000
Step 50000: w1 = 3.0000, w2 = 2.0000, b = 0, error = 0.0000
Step 75000: w1 = 3.0000, w2 = 2.0000, b = 0, error = 0.0000
Step 100000: w1 = 3.0000, w2 = 2.0000, b = 0, error = 0.0000
Step 125000: w1 = 3.0000, w2 = 2.0000, b = 0, error = 0.0000
Step 150000: w1 = 3.0000, w2 = 2.0000, b = 0, error = 0.0000
Step 175000: w1 = 3.0000, w2 = 2.0000, b = 0, error = 0.0000
Step 200000: w1 = 3.0000, w2 = 2.0000, b = 0, error = 0.0000
Step 225000: w1 = 3.0000, w2 = 2.0000, b = 0, error = 0.0000
Best weights ([w1,w2]) found: [2.999999696476301, 1.999999748484735]
Best bias (b) found: 0.0000
Mean squared error at best parameters: 0.0000
```